In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

df = pd.read_csv(r"C:\Users\Nisha\Downloads\global_retail.csv")
print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

In [3]:
```
import seaborn as sns
print("seaborn is ready")
```

```
seaborn is ready
```

In [ ]:
```
# Performing Data-Cleaning first!
```

In [4]:
```
display(df.head(10))
```

	price_record_id	product_code	product_name	brand	category	price	currency
0	339639	NaN	APFEL ROT LOSE	NaN	en:apples	2.08	EUR
1	339638	4061461467661	Orient. Salat 200g	NaN	NaN	1.29	EUR
2	339637	NaN	Toskanabrot 360g	NaN	en:white-breads	1.80	EUR
3	339636	4061464512245	Waffelhörn. Erdb.	NaN	NaN	0.49	EUR
4	339635	4061458026819	Orient. Salat 200g	NaN	NaN	1.29	EUR
5	339634	4061461866518	NaN	NaN	NaN	0.49	EUR
6	339633	4061464955394	Milchbrötchen	NaN	NaN	1.85	EUR
7	339632	4061458024068	Hummus 200 g	NaN	NaN	1.69	EUR
8	339630	4099200057439	Schokolade 100g	NaN	NaN	0.89	EUR
9	339631	4099200057378	Noisette 100g	NaN	NaN	1.25	EUR

In [5]:
```
print(df.columns.tolist())
```

```
['price_record_id', 'product_code', 'product_name', 'brand', 'category',
'price', 'currency', 'price_is_discounted', 'store_name', 'city', 'country
_code', 'date_recorded']
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 12 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   price_record_id      2000 non-null   int64  
 1   product_code         1866 non-null   object 
 2   product_name         1539 non-null   object 
 3   brand                0 non-null      float64
 4   category             134 non-null    object 
 5   price                2000 non-null   float64
 6   currency             2000 non-null   object 
 7   price_is_discounted  2000 non-null   bool   
 8   store_name           2000 non-null   object 
 9   city                 2000 non-null   object 
 10  country_code         0 non-null      float64
 11  date_recorded        2000 non-null   object 
dtypes: bool(1), float64(3), int64(1), object(7)
memory usage: 174.0+ KB
```

In [7]:
```
display(df.describe(include="all").T)
```

	count	unique	top	freq	mean	std
price_record_id	2000.0	NaN	NaN	NaN	338628.3005	587.603024
product_code	1866	1761	3770016162043	5	NaN	NaN
product_name	1539	1516	200ML SAUCE NUOC MAM AYAM	2	NaN	NaN
brand	0.0	NaN	NaN	NaN	NaN	NaN
category	134	64	en:breads	12	NaN	NaN
price	2000.0	NaN	NaN	NaN	34.239621	206.362881
currency	2000	20	EUR	994	NaN	NaN
price_is_discounted	2000	1	False	2000	NaN	NaN
store_name	2000	63	poslovitch	502	NaN	NaN
city	2000	79	Dumbo Market	289	NaN	NaN
country_code	0.0	NaN	NaN	NaN	NaN	NaN
date_recorded	2000	51	2026-10-02	446	NaN	NaN

In [8]:
```
missing = pd.DataFrame({
    "Missing_Count": df.isnull().sum(),
    "Missing_Percentage": (df.isnull().mean()*100).round(2)
})
display(missing.sort_values(
    by="Missing_Percentage",
    ascending=False
))
```

	Missing_Count	Missing_Percentage
brand	2000	100.00
country_code	2000	100.00
category	1866	93.30
product_name	461	23.05
product_code	134	6.70
price_record_id	0	0.00
currency	0	0.00
price	0	0.00
price_is_discounted	0	0.00
store_name	0	0.00
city	0	0.00
date_recorded	0	0.00

In [9]:
```
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

In [10]:
```
for col in df.columns:
    print(f"\ncolumn: {col}")
    print("Unique values:", df[col].nunique())
```

```
column: price_record_id
Unique values: 2000

column: product_code
Unique values: 1761

column: product_name
Unique values: 1516

column: brand
Unique values: 0

column: category
Unique values: 64

column: price
Unique values: 608

column: currency
Unique values: 20

column: price_is_discounted
Unique values: 1

column: store_name
Unique values: 63

column: city
Unique values: 79

column: country_code
Unique values: 0

column: date_recorded
Unique values: 51
```

In [11]:
```
display(df.select_dtypes(include=np.number).describe().T)
```

	count	mean	std	min	25%	50%
price_record_id	2000.0	338628.300500	587.603024	337609.00	338121.75	338631.50
brand	0.0	NaN	NaN	NaN	NaN	NaN
price	2000.0	34.239621	206.362881	0.23	2.28	4.37
country_code	0.0	NaN	NaN	NaN	NaN	NaN

In [ ]:
```
# Data cleaning
# Data Quality Assessment
In this section, we investigate missing values, duplicate records,
inconsistent categorical values, incorrect data types and
potetially invalid numerical observations.
```

In [12]:
```
# Missing value analysis
missing_report = pd.DataFrame({
    "Missing_count": df.isnull().sum(),
    "Missing_percentage": (df.isnull().mean()*100).round(2),
    "Data_Type": df.dtypes
```

```
})

display(missing_report.sort_values(
    by="Missing_percentage",
    ascending=False
))
```

	Missing_count	Missing_percentage	Data_Type
brand	2000	100.00	float64
country_code	2000	100.00	float64
category	1866	93.30	object
product_name	461	23.05	object
product_code	134	6.70	object
price_record_id	0	0.00	int64
currency	0	0.00	object
price	0	0.00	float64
price_is_discounted	0	0.00	bool
store_name	0	0.00	object
city	0	0.00	object
date_recorded	0	0.00	object

In [13]:
```
# Duplicate record analysis
print("Total records:", len(df))
print("Duplicate records:", df.duplicated().sum())
print("Percentage of duplicates:",
      round(df.duplicated().mean()*100, 2), "%")
```

```
Total records: 2000
Duplicate records: 0
Percentage of duplicates: 0.0 %
```

In [15]:
```
# Idnetify inconsistent categorical values
categorical_columns = df.select_dtypes(
    include=["object", "category"]
).columns

for col in categorical_columns:
    print(f"\n--- {col} ---")
    print(df[col].value_counts(dropna=False).head(20))
```

```
--- product_code ---
product_code
NaN              134
3770016162043      5
3760324300138      4
3760324300336      4
3272770093286      3
3770016162098      3
3760324300121      3
7649990821320      3
7032069761090      3
3497917003564      3
3770018934747      3
3497917000907      3
7034280109996      3
3760324300312      2
7032069731246      2
0075002120162      2
7032069101599      2
3760049795639      2
0076840485116      2
3760367390974      2
Name: count, dtype: int64

--- product_name ---
product_name
NaN                                                461
200ML SAUCE NUOC MAM AYAM                            2
WHOLESOME Organic Raw Unfiltered Honey 16 OZ         2
Orient. Salat 200g                                   2
ACCRO MERGUEZ 200G                                   2
PLANTED EFFILOCHE BBQ 160G                           2
BOULANG PAINS PANINI X4 300G LA BOULANGERE           2
PLANTED KEBAB 160G                                   2
HAPPYVORE KNAX 200G                                  2
JACQ BURG GEANT COMPLE 330G JACQUET                  2
NEMS HACHE VEGETAL 120G                              2
Fruchtjoghurt 180g                                   2
SØRLANDSCHIPS VERDENS TYNNESTE HAVSALT 230G          2
HPYRE TRANCHE TOMATE HERBES                          2
GALET.PANEE POIREAUX 200G MPX                        2
CROQ FOND FROM/EP X2 HAPPYVORE 180 g                 2
YT PROTEINSHAKE 300ML SJOKOLADE TINE                 2
PROTEIN SHAKE 250ML CARAMEL LATTE MAXIM              2
OLD BAY SEASONING 6 OZ                               2
CHAVROUX 9P 162G                                     2
Name: count, dtype: int64

--- category ---
category
NaN                     1866
en:breads                 12
en:microwave-meals         9
en:pastries                7
en:yogurts                 6
en:ice-creams              5
en:burek                   5
en:bananas                 4
en:pates                   4
en:pizzas                  4
```

```
en:soups                   3
en:chips                   3
en:sodas                   3
en:salty-snacks            3
en:edam                    2
en:kefirs                  2
en:fruit-juices            2
en:waters                  2
en:breakfast-cereals       2
en:raspberries             2
Name: count, dtype: int64

--- currency ---
currency
EUR    994
USD    407
NOK    190
BAM    122
SEK    118
CZK     73
KZT     35
PLN     21
SGD     16
ILS      5
RUB      4
INR      3
MAD      3
GBP      2
BIF      2
CLP      1
RSD      1
BRL      1
THB      1
HUF      1
Name: count, dtype: int64

--- store_name ---
store_name
poslovitch         502
doby               395
sebleouf           193
odinh              190
slavendjervida     122
freso              117
jbieber            105
tartelette299       52
stephane            36
vanished            35
jangop              22
monsieurtanuki      18
k127                18
therealestmca       17
guillaume029371     16
lpdj                14
piterpan2           12
hiimdannyganz       11
raphael0202         11
saraho888           11
Name: count, dtype: int64
```

```
--- city ---
city
Dumbo Market                  289
E. Leclerc                    214
Carrefour Market              202
Colruyt                       151
Monoprix                      128
Willys Landskrona Infarten    111
Intermarché                    87
Extra Danmarksplass            70
Whole Foods Market             67
Tropic                         64
Albert                         60
Rema 1000 Danmarksplass        58
Carrefour City                 52
Kiwi Bøhmergaten               44
Small                          35
Moj Market                     33
Đurić Market                   25
Smart & Final extra!           20
Hofer                          19
Rewe                           18
Name: count, dtype: int64

--- date_recorded ---
date_recorded
2026-10-02    446
2026-10-03    419
2026-10-01    317
2026-09-12    290
2026-09-30    181
2026-09-06    101
2026-09-29     48
2026-09-27     24
2026-05-31     22
2026-02-07     15
2026-09-21     12
2026-09-25     12
2026-05-15      9
2026-05-27      8
2026-05-25      7
2026-09-05      6
2026-05-10      5
2026-05-23      5
2026-06-07      5
2026-05-16      4
Name: count, dtype: int64
```

In [16]:

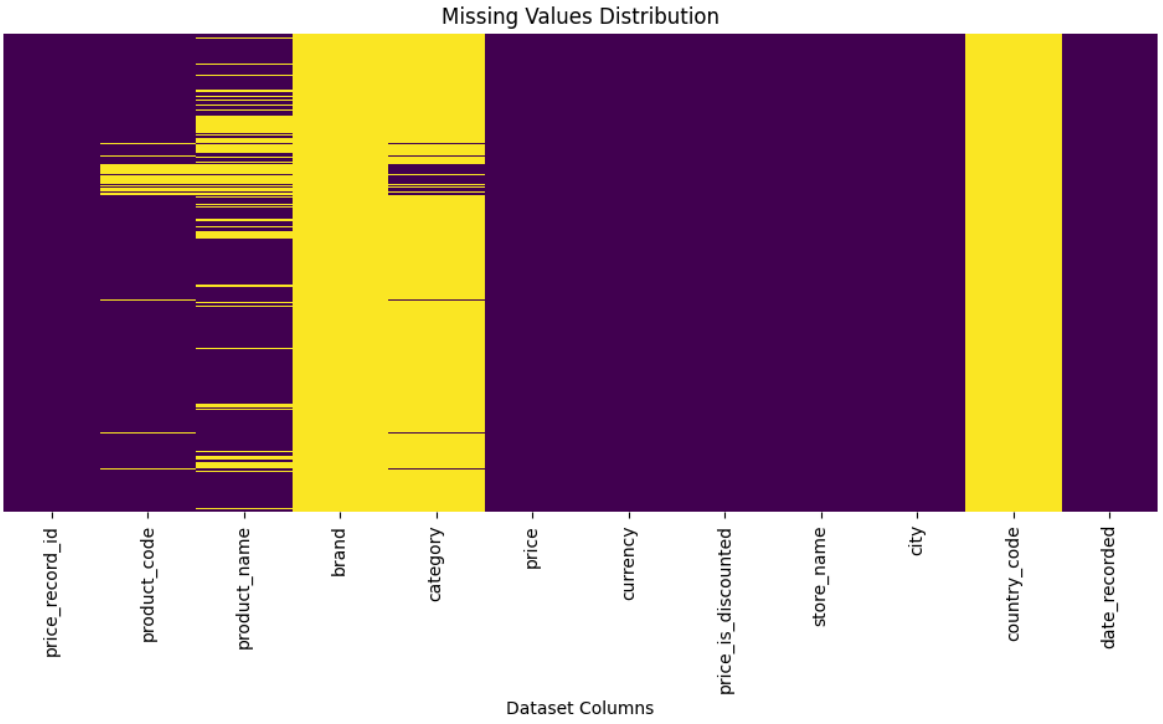
```python
# Identify numerical anomalies
numerical_columns = df.select_dtypes(
    include=np.number
).columns
display(df[numerical_columns].describe().T)
```

	count	mean	std	min	25%	50%
price_record_id	2000.0	338628.300500	587.603024	337609.00	338121.75	338631.50
brand	0.0	NaN	NaN	NaN	NaN	NaN
price	2000.0	34.239621	206.362881	0.23	2.28	4.37
country_code	0.0	NaN	NaN	NaN	NaN	NaN

In [17]:
```
# Visualize missing values
plt.figure(figsize=(12, 5))

sns.heatmap(
    df.isnull(),
    cbar=False,
    yticklabels=False,
    cmap="viridis"
)

plt.title("Missing Values Distribution")
plt.xlabel("Dataset Columns")
plt.show()
```



In [18]:
```
# Investigate columns with zero or very few unique values

for col in ["brand", "country_code", "price_is_discounted"]:

    print(f"\nColumn: {col}")

    print("Data type:", df[col].dtype)

    print("Missing values:", df[col].isnull().sum())

    print("Non-missing values:", df[col].count())

    print("Unique values including NaN:",
```

```
          df[col].nunique(dropna=False))

    print("Value distribution:")
    print(df[col].value_counts(dropna=False))
```

```
Column: brand
Data type: float64
Missing values: 2000
Non-missing values: 0
Unique values including NaN: 1
Value distribution:
brand
NaN    2000
Name: count, dtype: int64

Column: country_code
Data type: float64
Missing values: 2000
Non-missing values: 0
Unique values including NaN: 1
Value distribution:
country_code
NaN    2000
Name: count, dtype: int64

Column: price_is_discounted
Data type: bool
Missing values: 0
Non-missing values: 2000
Unique values including NaN: 1
Value distribution:
price_is_discounted
False    2000
Name: count, dtype: int64
```

In [19]:
```
# check whether product names and codes are consistent
product_name_variations = (
    df.groupby("product_code")["product_name"]
      .nunique()
      .sort_values(ascending=False)
)

print("Product codes with multiple product names:")

display(product_name_variations[
    product_name_variations > 1
].head(20))
```

```
Product codes with multiple product names:
```

```
product_code
3770016162043    4
3497917000907    3
3497917003564    3
3760324300336    3
7032069761090    3
7034280109996    3
3272770004725    2
3029330801931    2
0076840100989    2
0180626000599    2
0041736001909    2
3272770016131    2
8720182473813    2
0076840485116    2
3250390989516    2
0076840076574    2
3272770093286    2
3256540000339    2
0075002120162    2
0075002120247    2
Name: product_name, dtype: int64
```

In [21]:
```python
# check price and currency
print("Currency distribution:")
display(df["currency"].value_counts(dropna=False))
print("\nPrice summary:")
display(df["price"].describe())
print("\nNegative prices:", (df["price"] < 0).sum())
print("Zero prices:", (df["price"] == 0).sum())
```

```
Currency distribution:
currency
EUR    994
USD    407
NOK    190
BAM    122
SEK    118
CZK     73
KZT     35
PLN     21
SGD     16
ILS      5
RUB      4
INR      3
MAD      3
GBP      2
BIF      2
CLP      1
RSD      1
BRL      1
THB      1
HUF      1
Name: count, dtype: int64
Price summary:
```

```
count    2000.000000
mean       34.239621
std       206.362881
min         0.230000
25%         2.280000
50%         4.370000
75%        15.990000
max      5030.000000
Name: price, dtype: float64
Negative prices: 0
Zero prices: 0
```

In [27]:
```
print("Date data type:", df["date_recorded"].dtype)
print("\nDate distribution:")
display(df["date_recorded"].value_counts().head(15))
```

```
Date data type: object

Date distribution:
date_recorded
2026-10-02    446
2026-10-03    419
2026-10-01    317
2026-09-12    290
2026-09-30    181
2026-09-06    101
2026-09-29     48
2026-09-27     24
2026-05-31     22
2026-02-07     15
2026-09-21     12
2026-09-25     12
2026-05-15      9
2026-05-27      8
2026-05-25      7
Name: count, dtype: int64
```

In [28]:
```
# Create a working copy
df_clean = df.copy()

# Convert date column
df_clean["date_recorded"] = pd.to_datetime(
    df_clean["date_recorded"],
    errors="coerce"
)

# Verify conversion
print("New data type:",
      df_clean["date_recorded"].dtype)

print("Invalid dates:",
      df_clean["date_recorded"].isna().sum())
```

```
New data type: datetime64[ns]
Invalid dates: 0
```

In [30]:
```
df_clean["record_year"] = (
    df_clean["date_recorded"].dt.year
)

df_clean["record_month"] = (
```

```
    df_clean["date_recorded"].dt.month
)

df_clean["record_day"] = (
    df_clean["date_recorded"].dt.day
)
```

In [31]:
```
display(
    df_clean[
        ["date_recorded", "record_year",
         "record_month", "record_day"]
    ].head(10)
)
```

	date_recorded	record_year	record_month	record_day
0	2026-10-03	2026	10	3
1	2026-10-03	2026	10	3
2	2026-10-03	2026	10	3
3	2026-10-03	2026	10	3
4	2026-10-03	2026	10	3
5	2026-10-03	2026	10	3
6	2026-10-03	2026	10	3
7	2026-10-03	2026	10	3
8	2026-10-03	2026	10	3
9	2026-10-03	2026	10	3

In [32]:
```
# investigate the missing value
missing_report = pd.DataFrame({
    "Missing_Count": df_clean.isnull().sum(),
    "Missing_Percentage": (
        df_clean.isnull().mean() * 100
    ).round(2)
})

display(
    missing_report.sort_values(
        by="Missing_Percentage",
        ascending=False
    )
)
```

	Missing_Count	Missing_Percentage
brand	2000	100.00
country_code	2000	100.00
category	1866	93.30
product_name	461	23.05
product_code	134	6.70
price	0	0.00
price_record_id	0	0.00
currency	0	0.00
price_is_discounted	0	0.00
store_name	0	0.00
city	0	0.00
date_recorded	0	0.00
record_year	0	0.00
record_month	0	0.00
record_day	0	0.00

In [33]:
```
# Check non-missing category values
print("Available category records:")

display(
    df_clean["category"]
    .value_counts(dropna=False)
    .head(20)
)

# Check discount column
print("\nDiscount column values:")

display(
    df_clean["price_is_discounted"]
    .value_counts(dropna=False)
)

# Check currencies
print("\nCurrency distribution:")

display(
    df_clean["currency"]
    .value_counts()
)
```

```
Available category records:
```

```
category
NaN                     1866
en:breads                 12
en:microwave-meals         9
en:pastries                7
en:yogurts                 6
en:ice-creams              5
en:burek                   5
en:bananas                 4
en:pates                   4
en:pizzas                  4
en:soups                   3
en:chips                   3
en:sodas                   3
en:salty-snacks            3
en:edam                    2
en:kefirs                  2
en:fruit-juices            2
en:waters                  2
en:breakfast-cereals       2
en:raspberries             2
Name: count, dtype: int64
Discount column values:
price_is_discounted
False    2000
Name: count, dtype: int64
Currency distribution:
currency
EUR    994
USD    407
NOK    190
BAM    122
SEK    118
CZK     73
KZT     35
PLN     21
SGD     16
ILS      5
RUB      4
INR      3
MAD      3
GBP      2
BIF      2
CLP      1
RSD      1
BRL      1
THB      1
HUF      1
Name: count, dtype: int64
```

In [34]:

```
# Check products with missing names but available codes

display(
    df_clean.loc[
        df_clean["product_name"].isna()
        & df_clean["product_code"].notna(),
        ["product_code", "product_name", "category"]
    ].head(20)
)
```

	product_code	product_name	category
5	4061461866518	NaN	NaN
19	7394376623769	NaN	NaN
20	7394376624094	NaN	NaN
21	5411188119753	NaN	NaN
128	7340083453106	NaN	NaN
168	7394376615719	NaN	NaN
175	7394376620683	NaN	NaN
228	7340083471070	NaN	NaN
237	7340083494406	NaN	NaN
241	7394376614804	NaN	NaN
242	7394376623806	NaN	NaN
243	7394376624100	NaN	NaN
244	7803403004483	NaN	NaN
245	4335619098961	NaN	NaN
246	3664346348706	NaN	NaN
247	8003340092690	NaN	NaN
249	8719200311886	NaN	NaN
250	8719200295537	NaN	NaN
251	8719200296015	NaN	NaN
256	7394376622649	NaN	NaN

In [35]:
```
# Find product codes that have at least one known product name

known_products = (
    df_clean.dropna(subset=["product_name"])
    .groupby("product_code")["product_name"]
    .nunique()
)

print("Product codes with known names:",
      len(known_products))
```

```
Product codes with known names: 1436
```

In [36]:
```
# Product codes with missing names
missing_name_codes = df_clean.loc[
    df_clean["product_name"].isna()
    & df_clean["product_code"].notna(),
    "product_code"
].unique()
```

```
# Check which missing-name product codes exist
# among records with known names

recoverable_codes = [
    code for code in missing_name_codes
    if code in known_products.index
]

print("Missing product names:", len(missing_name_codes))

print("Potentially recoverable:", len(recoverable_codes))
```

```
Missing product names: 336
Potentially recoverable: 11
```

In [37]:

```
print("Total category records:",
      df_clean["category"].count())

print("Unique categories:",
      df_clean["category"].nunique())

print("\nAvailable categories:")

display(df_clean["category"].value_counts())
```

```
Total category records: 134
Unique categories: 64

Available categories:
```

category	
en:breads	12
en:microwave-meals	9
en:pastries	7
en:yogurts	6
en:ice-creams	5
en:burek	5
en:pates	4
en:bananas	4
en:pizzas	4
en:chips	3
en:soups	3
en:sodas	3
en:salty-snacks	3
en:fruit-juices	2
en:kefirs	2
en:cabbages	2
en:edam	2
en:breakfast-cereals	2
en:raspberries	2
en:tomatoes	2
en:smoked-meats	2
en:sausages	2
en:eggs	2
en:cream-cheeses	2
en:waters	2
en:candy-chocolate-bars	2
en:sandwiches	2
en:french-fries	2
en:sports-drinks	1
en:spices	1
en:bread-rolls	1
en:apples	1
en:tortillas	1
en:cheese-spreads	1
en:pastas	1
en:creams	1
en:cakes	1
en:chicken-breasts	1
en:white-breads	1
en:doughnuts	1
en:ground-chicken-meat	1
en:ground-beef-meats	1
en:pork	1
en:beetroot	1
en:onions	1
en:grapes	1
en:vegetables	1
en:salads	1
en:croissants	1
en:chocolate-milks	1
en:beverages	1
en:chocolates	1
en:beers	1
en:energy-drinks	1
en:frankfurters	1
en:chocolate-spreads	1
en:ketchup	1
en:popcorn	1
en:butter-croissants	1

```
en:peanuts                  1
fr:pains-multicereales      1
en:black-grapes             1
en:scallions                1
en:cucumbers                1
Name: count, dtype: int64
```

In [38]:
```python
# Identify completely empty columns
empty_columns = [
    col for col in df_clean.columns
    if df_clean[col].isna().all()
]

print("Completely empty columns:", empty_columns)

# Remove them
df_clean = df_clean.drop(columns=empty_columns)

print("\nRemaining columns:")
print(df_clean.columns.tolist())
```

```
Completely empty columns: ['brand', 'country_code']

Remaining columns:
['price_record_id', 'product_code', 'product_name', 'category', 'price',
'currency', 'price_is_discounted', 'store_name', 'city', 'date_recorded',
'record_year', 'record_month', 'record_day']
```

In [39]:
```python
# Find product codes associated with exactly one product name

name_counts = (
    df_clean.dropna(subset=["product_code", "product_name"])
    .groupby("product_code")["product_name"]
    .nunique()
)

valid_codes = name_counts[name_counts == 1].index

# Create a reliable product-code-to-name mapping
product_mapping = (
    df_clean[
        df_clean["product_code"].isin(valid_codes)
    ]
    .dropna(subset=["product_name"])
    .drop_duplicates("product_code")
    .set_index("product_code")["product_name"]
)

# Fill only missing product names
missing_before = df_clean["product_name"].isna().sum()

df_clean["product_name"] = df_clean["product_name"].fillna(
    df_clean["product_code"].map(product_mapping)
)

missing_after = df_clean["product_name"].isna().sum()

print("Missing names before:", missing_before)
print("Missing names after:", missing_after)
print("Names recovered:", missing_before - missing_after)
```

```
Missing names before: 461
Missing names after: 453
Names recovered: 8
```

In [40]:
```python
# investigate missing categories
category_counts = (
    df_clean.dropna(subset=["product_code", "category"])
    .groupby("product_code")["category"]
    .nunique()
)

valid_category_codes = category_counts[
    category_counts == 1
].index

category_mapping = (
    df_clean[
        df_clean["product_code"].isin(valid_category_codes)
    ]
    .dropna(subset=["category"])
    .drop_duplicates("product_code")
    .set_index("product_code")["category"]
)

missing_before = df_clean["category"].isna().sum()

df_clean["category"] = df_clean["category"].fillna(
    df_clean["product_code"].map(category_mapping)
)

missing_after = df_clean["category"].isna().sum()

print("Categories missing before:", missing_before)
print("Categories missing after:", missing_after)
print("Categories recovered:", missing_before - missing_after)
```

```
Categories missing before: 1866
Categories missing after: 1866
Categories recovered: 0
```

In [41]:
```python
# investigate missing product codes
print("Missing product codes:",
      df_clean["product_code"].isna().sum())

display(
    df_clean.loc[
        df_clean["product_code"].isna(),
        ["product_name", "category", "price", "currency"]
    ].head(15)
)
```

```
Missing product codes: 134
```

	product_name	category	price	currency
0	APFEL ROT LOSE	en:apples	2.08	EUR
2	Toskanabrot 360g	en:white-breads	1.80	EUR
10	Bananen lose	en:bananas	1.59	EUR
16	Schoko Donut 55g	en:doughnuts	0.39	EUR
18	Kaisersemmel	en:bread-rolls	0.23	EUR
420	H FRESH CHICKEN BREAST	en:chicken-breasts	4.92	SGD
421	GRASS FED MINCED BEEF	en:ground-beef-meats	8.79	SGD
422	H FRESH CHICKEN MINCE	en:ground-chicken-meat	1.60	SGD
447	NaN	en:beetroot	265.00	KZT
460	NaN	en:cabbages	178.00	KZT
461	Картофель мытый пакет	en:pork	5030.00	KZT
462	NaN	en:onions	110.00	KZT
509	OG RED CABBAGE	en:cabbages	4.39	USD
513	OG GREEN SEEDLESS GRAPES	en:grapes	7.32	USD
546	NaN	en:spices	0.29	BAM

In [42]:
```
# Clean categorical text
text_columns = df_clean.select_dtypes(
    include=["object", "category"]
).columns

for col in text_columns:
    if col != "date_recorded":
        df_clean[col] = df_clean[col].apply(
            lambda x: x.strip() if isinstance(x, str) else x
        )

print("Text whitespace cleaning completed.")
```

```
Text whitespace cleaning completed.
```

In [43]:
```
# validate numerical data
print("Negative prices:",
      (df_clean["price"] < 0).sum())

print("Zero prices:",
      (df_clean["price"] == 0).sum())

print("\nPrice summary:")

display(df_clean["price"].describe())
```

```
Negative prices: 0
Zero prices: 0

Price summary:
```

```
count    2000.000000
mean       34.239621
std       206.362881
min         0.230000
25%         2.280000
50%         4.370000
75%        15.990000
max      5030.000000
Name: price, dtype: float64
```

In [44]:
```python
# investigate the discount column
print(df_clean["price_is_discounted"].value_counts())
```

```
price_is_discounted
False    2000
Name: count, dtype: int64
```

In [45]:
```python
# Identify columns containing only one distinct value
constant_columns = [
    col for col in df_clean.columns
    if df_clean[col].nunique(dropna=False) <= 1
]

print("Constant columns:", constant_columns)

# Remove constant columns
df_clean = df_clean.drop(columns=constant_columns)

print("\nRemaining columns:")
print(df_clean.columns.tolist())
```

```
Constant columns: ['price_is_discounted', 'record_year']

Remaining columns:
['price_record_id', 'product_code', 'product_name', 'category', 'price',
'currency', 'store_name', 'city', 'date_recorded', 'record_month', 'record
_day']
```

In [46]:
```python
# validate numerical values
print("Negative prices:", (df_clean["price"] < 0).sum())
print("Zero prices:", (df_clean["price"] == 0).sum())
print("\nPrice statistics:")
display(df_clean["price"].describe())
```

```
Negative prices: 0
Zero prices: 0

Price statistics:
count    2000.000000
mean       34.239621
std       206.362881
min         0.230000
25%         2.280000
50%         4.370000
75%        15.990000
max      5030.000000
Name: price, dtype: float64
```

In [47]:
```python
Q1 = df_clean["price"].quantile(0.25)
Q3 = df_clean["price"].quantile(0.75)
```

```
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df_clean[
    (df_clean["price"] < lower_bound) |
    (df_clean["price"] > upper_bound)
]

print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
print("Potential price outliers:", len(outliers))
```

```
Lower bound: -18.285
Upper bound: 36.555
Potential price outliers: 166
```

In [48]:

```
currency_summary = (
    df_clean.groupby("currency")
    .agg(
        record_count=("price", "count"),
        average_price=("price", "mean"),
        minimum_price=("price", "min"),
        maximum_price=("price", "max")
    )
    .sort_values("record_count", ascending=False)
)

display(currency_summary)
```

currency	record_count	average_price	minimum_price	maximum_price
EUR	994	3.199809	0.23	29.01
USD	407	10.877740	0.50	43.99
NOK	190	34.934368	9.90	139.90
BAM	122	4.353115	0.29	27.99
SEK	118	23.973644	5.90	61.46
CZK	73	73.130137	9.90	319.00
KZT	35	1047.314286	110.00	5030.00
PLN	21	9.796190	1.97	42.30
SGD	16	3.324562	1.60	8.79
ILS	5	6.340000	5.00	9.90
RUB	4	53.750000	45.00	70.00
MAD	3	9.000000	2.00	23.00
INR	3	129.333333	20.00	209.00
GBP	2	1.895000	1.79	2.00
BIF	2	2000.000000	2000.00	2000.00
BRL	1	6.990000	6.99	6.99
CLP	1	1390.000000	1390.00	1390.00
HUF	1	1299.000000	1299.00	1299.00
RSD	1	169.000000	169.00	169.00
THB	1	1090.000000	1090.00	1090.00

In [49]:
```
# check repeated products across stores
product_frequency = (
    df_clean.dropna(subset=["product_code"])
    .groupby("product_code")
    .agg(
        observations=("price", "count"),
        different_stores=("store_name", "nunique"),
        different_cities=("city", "nunique"),
        different_currencies=("currency", "nunique")
    )
    .sort_values("observations", ascending=False)
)

display(product_frequency.head(20))
```

product_code	observations	different_stores	different_cities	different_currencies
3770016162043	5	3	4	1
3760324300138	4	3	2	1
3760324300336	4	3	1	1
3497917003564	3	1	3	1
3770016162098	3	2	2	1
3272770093286	3	1	3	1
3770018934747	3	2	1	1
3497917000907	3	1	3	1
3760324300121	3	1	2	1
7032069761090	3	1	1	1
7034280109996	3	1	1	1
7649990821320	3	1	1	1
3497917003151	2	1	2	1
3256540000339	2	1	2	1
3502110005670	2	2	2	1
3350033434776	2	1	1	1
0041736001909	2	1	1	1
3023292259014	2	1	2	1
3497917003588	2	1	2	1
0022506125100	2	1	1	1

In [50]:
```
# Final cleaning report
print("=" * 45)
print("GLOBAL RETAIL - CLEANING SUMMARY")
print("=" * 45)

print("Original records:", len(df))

print("Cleaned records:", len(df_clean))

print("Remaining columns:", len(df_clean.columns))

print("\nMissing values:")
display(df_clean.isnull().sum())

print("\nData types:")
display(df_clean.dtypes)

print("\nDuplicate records:", df_clean.duplicated().sum())
```

```
==========================================
GLOBAL RETAIL - CLEANING SUMMARY
==========================================
Original records: 2000
Cleaned records: 2000
Remaining columns: 11

Missing values:
price_record_id       0
product_code        134
product_name        453
category           1866
price                 0
currency              0
store_name            0
city                  0
date_recorded         0
record_month          0
record_day            0
dtype: int64
Data types:
price_record_id             int64
product_code               object
product_name               object
category                   object
price                     float64
currency                   object
store_name                 object
city                       object
date_recorded      datetime64[ns]
record_month                int32
record_day                  int32
dtype: object
Duplicate records: 0
```

In [51]:

```
# final data investigation before KNN
print("Number of currencies:",
      df_clean["currency"].nunique())

display(
    df_clean["currency"].value_counts().to_frame(
        name="Record_Count"
    )
)
```

```
Number of currencies: 20
```

currency	Record_Count
EUR	994
USD	407
NOK	190
BAM	122
SEK	118
CZK	73
KZT	35
PLN	21
SGD	16
ILS	5
RUB	4
INR	3
MAD	3
GBP	2
BIF	2
CLP	1
RSD	1
BRL	1
THB	1
HUF	1

In [52]:
```
# price outliers
Q1 = df_clean["price"].quantile(0.25)
Q3 = df_clean["price"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Potential outliers:",
      ((df_clean["price"] < lower_bound) |
       (df_clean["price"] > upper_bound)).sum())

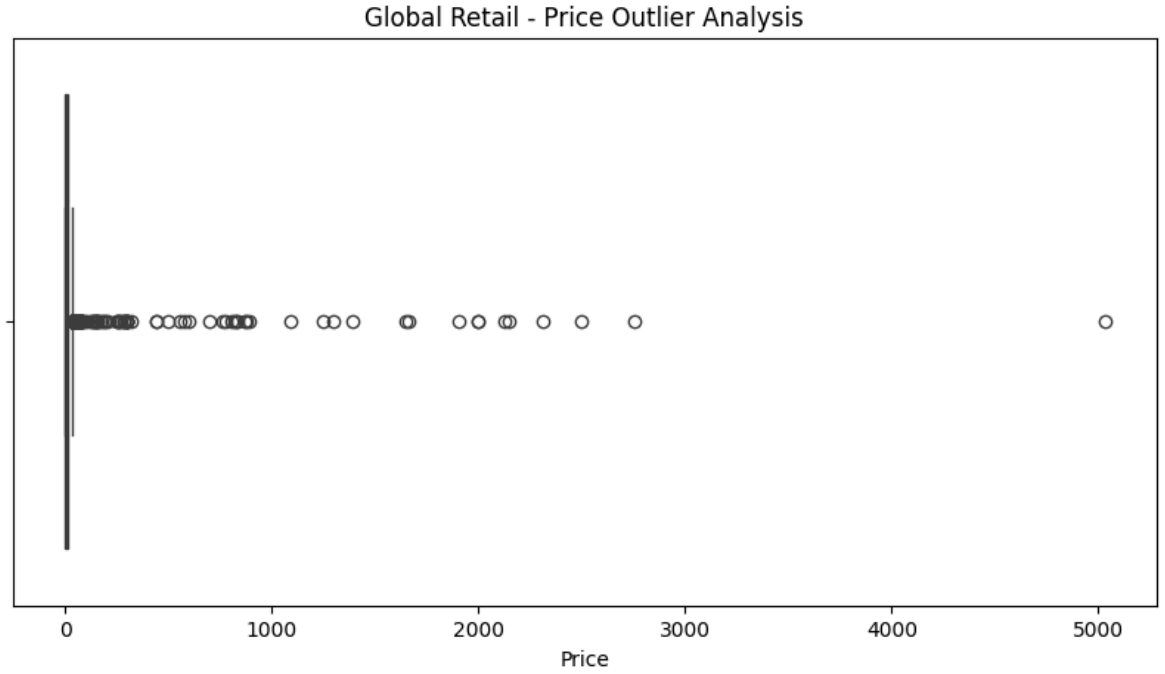
plt.figure(figsize=(10, 5))

sns.boxplot(x=df_clean["price"])

plt.title("Global Retail - Price Outlier Analysis")
```

```
plt.xlabel("Price")
plt.show()
```

```
Potential outliers: 166
```



In [53]:
```
# repeated product analysis
product_frequency = (
    df_clean.dropna(subset=["product_code"])
    .groupby("product_code")
    .agg(
        observations=("price", "count"),
        different_stores=("store_name", "nunique"),
        different_cities=("city", "nunique")
    )
    .sort_values("observations", ascending=False)
)

print("Products appearing multiple times:",
      (product_frequency["observations"] > 1).sum())

display(product_frequency.head(15))
```

```
Products appearing multiple times: 89
```

product_code	observations	different_stores	different_cities
3770016162043	5	3	4
3760324300138	4	3	2
3760324300336	4	3	1
3497917003564	3	1	3
3770016162098	3	2	2
3272770093286	3	1	3
3770018934747	3	2	1
3497917000907	3	1	3
3760324300121	3	1	2
7032069761090	3	1	1
7034280109996	3	1	1
7649990821320	3	1	1
3497917003151	2	1	2
3256540000339	2	1	2
3502110005670	2	2	2

In [54]:
```
print("Number of currencies:", df_clean["currency"].nunique())

display(
    df_clean["currency"]
    .value_counts()
    .to_frame(name="Record_Count")
)
```

```
Number of currencies: 20
```

currency	Record_Count
EUR	994
USD	407
NOK	190
BAM	122
SEK	118
CZK	73
KZT	35
PLN	21
SGD	16
ILS	5
RUB	4
INR	3
MAD	3
GBP	2
BIF	2
CLP	1
RSD	1
BRL	1
THB	1
HUF	1

In [55]:
```
# Create a separate dataset containing only EUR records
df_eur = df_clean[df_clean["currency"] == "EUR"].copy()

print("EUR dataset shape:", df_eur.shape)

# Basic price statistics
print("\nEUR price statistics:")
display(df_eur["price"].describe().to_frame())

# Check invalid prices
print("\nZero or negative prices:",
      (df_eur["price"] <= 0).sum())

# Check potential outliers using IQR
Q1 = df_eur["price"].quantile(0.25)
Q3 = df_eur["price"].quantile(0.75)
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df_eur[
    (df_eur["price"] < lower_bound) |
    (df_eur["price"] > upper_bound)
]

print("\nLower bound:", lower_bound)
print("Upper bound:", upper_bound)
print("Potential outliers:", len(outliers))

# Missing values in EUR records
print("\nMissing values:")
display(df_eur.isnull().sum().to_frame("Missing_Count"))

# Repeated products within EUR
print("\nRepeated product codes in EUR:",
      df_eur["product_code"].dropna().duplicated().sum())
```

```
EUR dataset shape: (994, 11)

EUR price statistics:
```

	price
count	994.000000
mean	3.199809
std	2.854417
min	0.230000
25%	1.790000
50%	2.505000
75%	3.690000
max	29.010000

```
Zero or negative prices: 0

Lower bound: -1.0599999999999996
Upper bound: 6.539999999999999
Potential outliers: 54

Missing values:
```

	Missing_Count
price_record_id	0
product_code	8
product_name	156
category	986
price	0
currency	0
store_name	0
city	0
date_recorded	0
record_month	0
record_day	0

```
Repeated product codes in EUR: 56
```

In [56]:

```python
# 1. Inspect the 15 highest-priced observations
print("Top 15 highest-priced EUR products:")

display(
    df_eur.sort_values("price", ascending=False)[
        ["product_code", "product_name", "price",
         "store_name", "city", "date_recorded"]
    ].head(15)
)


# 2. Count distinct products that appear more than once
product_counts = df_eur["product_code"].value_counts()

repeated_products = product_counts[product_counts > 1]

print("\nDistinct product codes appearing more than once:",
      len(repeated_products))

print("Total additional repeated observations:",
      (repeated_products - 1).sum())


# 3. Check the usefulness of potential features
print("\nUnique values in each potential feature:")

for col in ["product_name", "store_name", "city",
            "record_month", "record_day"]:
    print(col, ":", df_eur[col].nunique())


# 4. Inspect rows with missing product codes
print("\nRows with missing product codes:")

display(
```

```
    df_eur[df_eur["product_code"].isna()][
        ["product_name", "price", "store_name", "city"]
    ]
)
```

```
Top 15 highest-priced EUR products:
```

	product_code	product_name	price	store_name	city
489	2102612460019025001908 60	NaN	29.01	freemannameerf	Alcampo
1585	5601252110995	GALLO H.OLIVE EXTRA VIERG 40 2.000 L	25.87	poslovitch	Colruyt
1987	8006530143400	COUCHES BBDRY BIG PACK T4 X72 PAMPERS	25.39	poslovitch	Intermarché
122	3253959000189	AOP CHAMPAGNE GERMAIN BRUT 75cL	24.90	poslovitch	E. Leclerc
858	5011026108033	NaN	22.00	sebleouf	U Express Lyon Université
951	4067700013859	JAGERMEISTER 35%V LIQUEUR 70CL	21.29	sebleouf	Carrefour Market
857	3147699111154	NaN	20.66	sebleouf	U Express Lyon Université
922	8000070084698	1KG GRAINS MOMENTO VIVACE LVZ	19.50	sebleouf	Carrefour Market
859	5000267014203	NaN	19.08	sebleouf	U Express Lyon Université
1949	8006530132039	COUCHES PREMIUM PROTECT T2X PAMPERS	17.74	poslovitch	Intermarché
1948	8006530140638	COUCHES BABY DRY GEANT T5X39 PAMPERS	17.56	poslovitch	Intermarché
952	3020884441109	GET 31 PIPPERMINT 24%V BOUTEILLE 70CL	16.75	sebleouf	Carrefour Market
1615	8700216910194	41PANTS T4 PAMP.BABYDRY 40	16.70	poslovitch	Colruyt
1616	8700216910118	36 PANTS T5 PAMP.BABYDRY	16.59	poslovitch	Colruyt

	product_code	product_name	price	store_name	city
		40			
1125	8435223281097	NaN	15.99	ampeg	Grand Frais

```
Distinct product codes appearing more than once: 42
Total additional repeated observations: 56

Unique values in each potential feature:
product_name : 818
store_name : 40
city : 39
record_month : 2
record_day : 13

Rows with missing product codes:
```

	product_name	price	store_name	city
0	APFEL ROT LOSE	2.08	jbieber	Hofer
2	Toskanabrot 360g	1.80	jbieber	Hofer
10	Bananen lose	1.59	jbieber	Hofer
16	Schoko Donut 55g	0.39	jbieber	Hofer
18	Kaisersemmel	0.23	jbieber	Hofer
1115	Frz. Butter-Croissant	0.49	k127	Norma
1761	Raisin noir (famille)	5.95	raphael0202	Miyam
1762	Tomate RONDE (famille)	4.95	raphael0202	Miyam

In [ ]:
```
# KNN experiment
Predict the retail price of an EUR product using available product and re
- Target (y): price
- Candidate features (X): product_name, store_name, city
- Exclude: price_record_id, product_code, currency, date_recorded
- Exclude category because it is approximately 99.2% missing.
- Initially exclude record_month and record_day; there is too little temp
```

In [57]:
```
# 15 highest prices
display(
    df_eur.sort_values(
        "price", ascending=False
    )[
        ["product_name", "price", "store_name", "city"]
    ].head(15)
)
```

	product_name	price	store_name	city
489	NaN	29.01	freemannameerf	Alcampo
1585	GALLO H.OLIVE EXTRA VIERG 40 2.000 L	25.87	poslovitch	Colruyt
1987	COUCHES BBDRY BIG PACK T4 X72 PAMPERS	25.39	poslovitch	Intermarché
122	AOP CHAMPAGNE GERMAIN BRUT 75cL	24.90	poslovitch	E. Leclerc
858	NaN	22.00	sebleouf	U Express Lyon Université
951	JAGERMEISTER 35%V LIQUEUR 70CL	21.29	sebleouf	Carrefour Market
857	NaN	20.66	sebleouf	U Express Lyon Université
922	1KG GRAINS MOMENTO VIVACE LVZ	19.50	sebleouf	Carrefour Market
859	NaN	19.08	sebleouf	U Express Lyon Université
1949	COUCHES PREMIUM PROTECT T2X PAMPERS	17.74	poslovitch	Intermarché
1948	COUCHES BABY DRY GEANT T5X39 PAMPERS	17.56	poslovitch	Intermarché
952	GET 31 PIPPERMINT 24%V BOUTEILLE 70CL	16.75	sebleouf	Carrefour Market
1615	41PANTS T4 PAMP.BABYDRY 40	16.70	poslovitch	Colruyt
1616	36 PANTS T5 PAMP.BABYDRY 40	16.59	poslovitch	Colruyt
1125	NaN	15.99	ampeg	Grand Frais

In [58]:
```
display(
    df_eur[df_eur["product_code"].isna()][
        ["product_name", "price", "store_name", "city"]
    ]
)
```

	product_name	price	store_name	city
0	APFEL ROT LOSE	2.08	jbieber	Hofer
2	Toskanabrot 360g	1.80	jbieber	Hofer
10	Bananen lose	1.59	jbieber	Hofer
16	Schoko Donut 55g	0.39	jbieber	Hofer
18	Kaisersemmel	0.23	jbieber	Hofer
1115	Frz. Butter-Croissant	0.49	k127	Norma
1761	Raisin noir (famille)	5.95	raphael0202	Miyam
1762	Tomate RONDE (famille)	4.95	raphael0202	Miyam

In [59]:
```
# Select the features and target
features = ["product_name", "store_name", "city"]
target = "price"

# Create modelling dataset
df_model = df_eur[features + [target]].copy()

# Remove rows where target is missing
df_model = df_model.dropna(subset=[target])

# Check dataset
print("Modelling dataset shape:", df_model.shape)

print("\nMissing values:")
display(df_model.isnull().sum().to_frame("Missing_Count"))

print("\nSample observations:")
display(df_model.head(10))

print("\nData types:")
print(df_model.dtypes)
```

```
Modelling dataset shape: (994, 4)

Missing values:
```

	Missing_Count
product_name	156
store_name	0
city	0
price	0

```
Sample observations:
```

	product_name	store_name	city	price
0	APFEL ROT LOSE	jbieber	Hofer	2.08
1	Orient. Salat 200g	jbieber	Hofer	1.29
2	Toskanabrot 360g	jbieber	Hofer	1.80
3	Waffelhörn. Erdb.	jbieber	Hofer	0.49
4	Orient. Salat 200g	jbieber	Hofer	1.29
5	NaN	jbieber	Hofer	0.49
6	Milchbrötchen	jbieber	Hofer	1.85
7	Hummus 200 g	jbieber	Hofer	1.69
8	Schokolade 100g	jbieber	Hofer	0.89
9	Noisette 100g	jbieber	Hofer	1.25

```
Data types:
product_name     object
store_name       object
city             object
price           float64
dtype: object
```

In [62]:

```
from sklearn.model_selection import train_test_split
import sklearn

# Separate input features (X) and target (y)
X = df_model[["product_name", "store_name", "city"]]
y = df_model["price"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Scikit-learn version:", sklearn.__version__)

print("\nTraining features:", X_train.shape)
print("Testing features:", X_test.shape)

print("\nTraining target:", y_train.shape)
print("Testing target:", y_test.shape)

print("\nTraining price statistics:")
display(y_train.describe().to_frame())

print("\nTesting price statistics:")
display(y_test.describe().to_frame())
```

```
Scikit-learn version: 1.9.1

Training features: (795, 3)
Testing features: (199, 3)

Training target: (795,)
Testing target: (199,)

Training price statistics:
```

	price
count	795.000000
mean	3.172541
std	2.944618
min	0.230000
25%	1.790000
50%	2.490000
75%	3.590000
max	29.010000

```
Testing price statistics:
```

	price
count	199.000000
mean	3.308744
std	2.465023
min	0.770000
25%	1.750000
50%	2.690000
75%	3.950000
max	17.560000

In [61]:
```
%pip install scikit-learn
```

```
Collecting scikit-learnNote: you may need to restart the kernel to use upd
ated packages.

  Downloading scikit_learn-1.9.1-cp311-cp311-win_amd64.whl.metadata (9.3 k
B)
Requirement already satisfied: numpy>=1.24.1 in C:\python3.7\Lib\site-pack
ages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.10.0 in C:\python3.7\Lib\site-pack
ages (from scikit-learn) (1.15.1)
Collecting joblib>=1.4.0 (from scikit-learn)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting narwhals>=2.0.1 (from scikit-learn)
  Downloading narwhals-2.28.0-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-learn)
  Downloading threadpoolctl-3.7.0-py3-none-any.whl.metadata (24 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.4.0->scikit-learn)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Downloading scikit_learn-1.9.1-cp311-cp311-win_amd64.whl (8.3 MB)
   ---------------------------------------- 0.0/8.3 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.3 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.3 MB ? eta -:--:--
   --- ------------------------------------ 0.8/8.3 MB 3.0 MB/s eta 0:00:0
3
   -------- ------------------------------- 1.8/8.3 MB 4.2 MB/s eta 0:00:0
2
   ----------------- ---------------------- 3.9/8.3 MB 6.3 MB/s eta 0:00:0
1
   ---------------------------------- -- 7.9/8.3 MB 9.0 MB/s eta 0:00:0
1
   ---------------------------------------- 8.3/8.3 MB 8.9 MB/s  0:00:01
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading narwhals-2.28.0-py3-none-any.whl (490 kB)
Downloading threadpoolctl-3.7.0-py3-none-any.whl (26 kB)
Installing collected packages: threadpoolctl, narwhals, cloudpickle, jobli
b, scikit-learn

   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
   -------- ------------------------------- 1/5 [narwhals]
```

-------- ------------------------------ 1/5 [narwhals]
--------------- ---------------------- 2/5 [cloudpickle]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
----------------------- --------------- 3/5 [joblib]
-------------------------------- ------- 4/5 [scikit-learn]
-------------------------------- ------- 4/5 [scikit-learn]
-------------------------------- ------- 4/5 [scikit-learn

---------------------------------- ------- 4/5 [scikit-learn]
---------------------------------- ------- 4/5 [scikit-learn]
---------------------------------- ------- 4/5 [scikit-learn]

```
  -------------------------------- ------- 4/5 [scikit-learn]
  -------------------------------- ------- 4/5 [scikit-learn]
  ---------------------------------------- 5/5 [scikit-learn]

Successfully installed cloudpickle-3.1.2 joblib-1.6.0 narwhals-2.28.0 scik
it-learn-1.9.1 threadpoolctl-3.7.0
```

In [63]:

```python
# building my first KNN model
import numpy as np

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.neighbors import KNeighborsRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# All three input features are categorical
categorical_features = ["product_name", "store_name", "city"]

# Build preprocessing + KNN pipeline
knn_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="constant",
            fill_value="Unknown"
        )
    ),
    (
        "encoder",
        OneHotEncoder(handle_unknown="ignore")
    ),
    (
        "knn",
        KNeighborsRegressor(n_neighbors=5)
    )
])

# Train the KNN model
knn_pipeline.fit(X_train, y_train)

# Predict prices on unseen test observations
y_pred = knn_pipeline.predict(X_test)

# Evaluate KNN
knn_mae = mean_absolute_error(y_test, y_pred)
knn_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
knn_r2 = r2_score(y_test, y_pred)

print("KNN MODEL PERFORMANCE")
print("----------------------")
print("MAE:", round(knn_mae, 4))
print("RMSE:", round(knn_rmse, 4))
print("R² Score:", round(knn_r2, 4))
```

```
KNN MODEL PERFORMANCE
--------------------
MAE: 1.6876
RMSE: 2.6093
R² Score: -0.1262
```

In [64]:
```python
# compare against a baseline
# Baseline model: always predicts the training median
baseline = DummyRegressor(strategy="median")

baseline.fit(X_train, y_train)

baseline_pred = baseline.predict(X_test)

baseline_mae = mean_absolute_error(y_test, baseline_pred)
baseline_rmse = np.sqrt(
    mean_squared_error(y_test, baseline_pred)
)
baseline_r2 = r2_score(y_test, baseline_pred)

print("BASELINE MODEL PERFORMANCE")
print("--------------------------")
print("MAE:", round(baseline_mae, 4))
print("RMSE:", round(baseline_rmse, 4))
print("R² Score:", round(baseline_r2, 4))

print("\nKNN vs Baseline")
print("--------------------------")
print("KNN MAE:", round(knn_mae, 4))
print("Baseline MAE:", round(baseline_mae, 4))
```

```
BASELINE MODEL PERFORMANCE
--------------------------
MAE: 1.5496
RMSE: 2.5916
R² Score: -0.1109

KNN vs Baseline
--------------------------
KNN MAE: 1.6876
Baseline MAE: 1.5496
```

In [65]:
```python
# experiment 2: tune KNN instead of accepting k=5
from sklearn.model_selection import GridSearchCV

# Hyperparameter combinations
param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 11, 15, 20, 25, 30],
    "knn__weights": ["uniform", "distance"],
    "knn__p": [1, 2]
}

# Grid search using MAE
grid_search = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=param_grid,
    scoring="neg_mean_absolute_error",
    cv=5,
    n_jobs=-1,
    verbose=1
```

```
)

# Tune using training data only
grid_search.fit(X_train, y_train)

print("\nBest parameters:")
print(grid_search.best_params_)

print("\nBest cross-validation MAE:")
print(round(-grid_search.best_score_, 4))
```

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits

Best parameters:
{'knn__n_neighbors': 3, 'knn__p': 2, 'knn__weights': 'distance'}

Best cross-validation MAE:
1.2848
```

In [66]:
```
# Evaluate the tuned model
# Select the best model
best_knn = grid_search.best_estimator_

# Predict on the untouched test set
tuned_pred = best_knn.predict(X_test)

# Evaluation
tuned_mae = mean_absolute_error(y_test, tuned_pred)
tuned_rmse = np.sqrt(mean_squared_error(y_test, tuned_pred))
tuned_r2 = r2_score(y_test, tuned_pred)

print("TUNED KNN PERFORMANCE")
print("----------------------")
print("MAE:", round(tuned_mae, 4))
print("RMSE:", round(tuned_rmse, 4))
print("R² Score:", round(tuned_r2, 4))

print("\nMODEL COMPARISON")
print("----------------------")
print("Baseline MAE:", round(baseline_mae, 4))
print("Original KNN MAE:", round(knn_mae, 4))
print("Tuned KNN MAE:", round(tuned_mae, 4))
```

```
TUNED KNN PERFORMANCE
----------------------
MAE: 1.4119
RMSE: 2.4736
R² Score: -0.0121

MODEL COMPARISON
----------------------
Baseline MAE: 1.5496
Original KNN MAE: 1.6876
Tuned KNN MAE: 1.4119
```

In [67]:
```
# experiment 3: test whether KNN can predict prices for unseen products
from sklearn.model_selection import GroupShuffleSplit, GroupKFold, GridSe

# Recover product codes aligned with df_model
groups = df_eur.loc[df_model.index, "product_code"].copy()
```

```
# Give each missing product code a unique group
missing_mask = groups.isna()

groups = groups.astype("object")
groups.loc[missing_mask] = [
    f"MISSING_ROW_{idx}"
    for idx in groups.index[missing_mask]
]

groups = groups.astype(str)

# Create a product-grouped train/test split
group_split = GroupShuffleSplit(
    n_splits=1,
    test_size=0.20,
    random_state=42
)

train_idx, test_idx = next(
    group_split.split(df_model, df_model["price"], groups)
)

X_group = df_model[["product_name", "store_name", "city"]]
y_group = df_model["price"]

X_train_group = X_group.iloc[train_idx]
X_test_group = X_group.iloc[test_idx]

y_train_group = y_group.iloc[train_idx]
y_test_group = y_group.iloc[test_idx]

groups_train = groups.iloc[train_idx]

print("Grouped training shape:", X_train_group.shape)
print("Grouped testing shape:", X_test_group.shape)

# Verify that no product groups overlap
train_groups = set(groups.iloc[train_idx])
test_groups = set(groups.iloc[test_idx])

print("Overlapping product groups:", len(train_groups & test_groups))
```

```
Grouped training shape: (793, 3)
Grouped testing shape: (201, 3)
Overlapping product groups: 0
```

In [68]:

```
# Tune KNN using grouped cross-validation
group_cv = GroupKFold(n_splits=5)

group_grid = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=param_grid,
    scoring="neg_mean_absolute_error",
    cv=group_cv,
    n_jobs=-1,
    verbose=1
)

group_grid.fit(
    X_train_group,
```

```
    y_train_group,
    groups=groups_train
)

print("\nBest grouped parameters:")
print(group_grid.best_params_)

print("\nBest grouped CV MAE:")
print(round(-group_grid.best_score_, 4))
```

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits

Best grouped parameters:
{'knn__n_neighbors': 20, 'knn__p': 1, 'knn__weights': 'distance'}

Best grouped CV MAE:
1.5121
```

In [69]:
```
# Evaluate on unseen product groups
best_group_knn = group_grid.best_estimator_

group_pred = best_group_knn.predict(X_test_group)

group_mae = mean_absolute_error(y_test_group, group_pred)
group_rmse = np.sqrt(
    mean_squared_error(y_test_group, group_pred)
)
group_r2 = r2_score(y_test_group, group_pred)

# Baseline using training median
group_baseline = DummyRegressor(strategy="median")
group_baseline.fit(X_train_group, y_train_group)

group_baseline_pred = group_baseline.predict(X_test_group)

group_baseline_mae = mean_absolute_error(
    y_test_group, group_baseline_pred
)

print("UNSEEN-PRODUCT KNN PERFORMANCE")
print("--------------------------------")
print("MAE:", round(group_mae, 4))
print("RMSE:", round(group_rmse, 4))
print("R²:", round(group_r2, 4))

print("\nGROUPED BASELINE MAE:", round(group_baseline_mae, 4))
```

```
UNSEEN-PRODUCT KNN PERFORMANCE
--------------------------------
MAE: 1.3205
RMSE: 1.967
R²: -0.0729

GROUPED BASELINE MAE: 1.2812
```

In [70]:
```
# experiment 4A -- TF-IDF + KNN
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.neighbors import KNeighborsRegressor
```

```
from sklearn.model_selection import GridSearchCV, GroupKFold

# Prepare text input; replace missing names with empty strings
X_train_text = X_train_group.copy()
X_test_text = X_test_group.copy()

X_train_text["product_name"] = (
    X_train_text["product_name"].fillna("")
)
X_test_text["product_name"] = (
    X_test_text["product_name"].fillna("")
)

# Text and location preprocessing
text_preprocessor = ColumnTransformer(
    transformers=[
        (
            "product_text",
            TfidfVectorizer(
                ngram_range=(1, 2),
                strip_accents="unicode"
            ),
            "product_name"
        ),
        (
            "location",
            OneHotEncoder(handle_unknown="ignore"),
            ["store_name", "city"]
        )
    ]
)

# Build the new KNN pipeline
tfidf_knn = Pipeline([
    ("preprocessor", text_preprocessor),
    (
        "knn",
        KNeighborsRegressor(
            metric="cosine",
            algorithm="brute",
            weights="distance"
        )
    )
])

# Tune the number of neighbours
tfidf_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 10, 15, 20, 30]
}

# Grouped cross-validation
tfidf_search = GridSearchCV(
    estimator=tfidf_knn,
    param_grid=tfidf_param_grid,
    scoring="neg_mean_absolute_error",
    cv=GroupKFold(n_splits=5),
    n_jobs=-1,
    verbose=1
)
```

```
tfidf_search.fit(
    X_train_text,
    y_train_group,
    groups=groups_train
)

print("Best TF-IDF KNN parameters:")
print(tfidf_search.best_params_)

print(
    "\nBest grouped CV MAE:",
    round(-tfidf_search.best_score_, 4)
)
```

```
Fitting 5 folds for each of 7 candidates, totalling 35 fits
Best TF-IDF KNN parameters:
{'knn__n_neighbors': 3}

Best grouped CV MAE: 1.1314
```

In [71]:
```
# experiment 4B -- Evaluate on the same unseen-product test set
# Best TF-IDF KNN model
best_tfidf_knn = tfidf_search.best_estimator_

# Predictions
tfidf_pred = best_tfidf_knn.predict(X_test_text)

# Metrics
tfidf_mae = mean_absolute_error(y_test_group, tfidf_pred)

tfidf_rmse = np.sqrt(
    mean_squared_error(y_test_group, tfidf_pred)
)

tfidf_r2 = r2_score(y_test_group, tfidf_pred)

print("TF-IDF + KNN PERFORMANCE")
print("-------------------------")
print("MAE:", round(tfidf_mae, 4))
print("RMSE:", round(tfidf_rmse, 4))
print("R²:", round(tfidf_r2, 4))

print("\nCOMPARISON WITH GROUPED BASELINE")
print("--------------------------------")
print("Baseline MAE:", round(group_baseline_mae, 4))
print("Previous KNN MAE:", round(group_mae, 4))
print("TF-IDF KNN MAE:", round(tfidf_mae, 4))
```

```
TF-IDF + KNN PERFORMANCE
-------------------------
MAE: 0.9785
RMSE: 1.8303
R²: 0.071

COMPARISON WITH GROUPED BASELINE
--------------------------------
Baseline MAE: 1.2812
Previous KNN MAE: 1.3205
TF-IDF KNN MAE: 0.9785
```

In [ ]:
```
One important detail of my dataset: product_name, store_name and city are
We cannot feed them directly into Linear Regression. We'll transform them
Step1: Prepare the grouped training and testing data.
```

In [72]:
```
# Check that our grouped split variables are available

print("Training features:", X_train_group.shape)
print("Testing features:", X_test_group.shape)

print("\nTraining target:", y_train_group.shape)
print("Testing target:", y_test_group.shape)

print("\nTraining feature columns:")
print(X_train_group.columns.tolist())

print("\nMissing values in training data:")
print(X_train_group.isnull().sum())

print("\nMissing values in testing data:")
print(X_test_group.isnull().sum())
```

```
Training features: (793, 3)
Testing features: (201, 3)

Training target: (793,)
Testing target: (201,)

Training feature columns:
['product_name', 'store_name', 'city']

Missing values in training data:
product_name    135
store_name        0
city              0
dtype: int64

Missing values in testing data:
product_name    21
store_name       0
city             0
dtype: int64
```

In [73]:
```
# step2: Build the Preprocessing pipeline
import numpy as np

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, FunctionTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LinearRegression

# 1. Process product names
product_text_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="")),
    ("flatten", FunctionTransformer(np.ravel, feature_names_out="one-to-o
    ("tfidf", TfidfVectorizer(
        ngram_range=(1, 2),
        strip_accents="unicode",
```

```
        lowercase=True
    ))
])

# 2. Process store and city
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# 3. Combine both preprocessing branches
preprocessor = ColumnTransformer([
    ("product_text", product_text_pipeline, ["product_name"]),
    ("categorical", categorical_pipeline, ["store_name", "city"])
])

# 4. Create the complete Linear Regression pipeline
linear_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

print("Linear Regression pipeline created successfully!")
print(linear_pipeline)
```

```
Linear Regression pipeline created successfully!
Pipeline(steps=[('preprocessor',
                 ColumnTransformer(transformers=[('product_text',
                                                  Pipeline(steps=[('impute
r',
                                                                   SimpleI
mputer(fill_value='',

strategy='constant')),
                                                                  ('flatte
n',
                                                                   Functio
nTransformer(feature_names_out='one-to-one',

func=<function ravel at 0x000002A7FEDAC830>)),
                                                                  ('tfid
f',
                                                                   TfidfVe
ctorizer(ngram_range=(1,

2),

strip_accents='unicode'))]),
                                                  ['product_name']),
                                                 ('categorical',
                                                  Pipeline(steps=[('impute
r',
                                                                   SimpleI
mputer(strategy='most_frequent')),
                                                                  ('encode
r',
                                                                   OneHotE
ncoder(handle_unknown='ignore'))]),
                                                  ['store_name', 'cit
y'])])),
                ('regressor', LinearRegression())])
```

In [74]:
```
# step3 -- Train the Linear Regression model
# Train the Linear Regression pipeline
linear_pipeline.fit(X_train_group, y_train_group)
print("Linear Regression model trained successfully!")
```

```
Linear Regression model trained successfully!
```

In [75]:
```
# Make predictions and evaluate
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_s

# Predict prices on unseen test data
y_pred_linear = linear_pipeline.predict(X_test_group)

# Calculate evaluation metrics
linear_mae = mean_absolute_error(y_test_group, y_pred_linear)
linear_rmse = np.sqrt(mean_squared_error(y_test_group, y_pred_linear))
linear_r2 = r2_score(y_test_group, y_pred_linear)

print("LINEAR REGRESSION - GROUPED TEST RESULTS")
print("-" * 45)
print(f"MAE  : {linear_mae:.4f}")
print(f"RMSE : {linear_rmse:.4f}")
print(f"R²   : {linear_r2:.4f}")
```

```
LINEAR REGRESSION - GROUPED TEST RESULTS
--------------------------------------------
MAE  : 1.0206
RMSE : 1.5082
R²   : 0.3693
```

In [76]:
```python
# step6: check whether the result is reliable
from sklearn.model_selection import GroupKFold, cross_validate

# Reconstruct product groups using the original product codes
groups_all = df_eur.loc[df_model.index, "product_code"].copy()

# Give missing product codes unique group identifiers
groups_all = groups_all.astype("object")
missing_mask = groups_all.isna()

groups_all.loc[missing_mask] = [
    f"MISSING_ROW_{idx}"
    for idx in groups_all.index[missing_mask]
]

groups_all = groups_all.astype(str)

# Keep only groups belonging to the training observations
groups_train_lr = groups_all.loc[X_train_group.index]

print("Training observations:", len(X_train_group))
print("Training groups:", groups_train_lr.nunique())
print("Missing group identifiers:", groups_train_lr.isna().sum())
```

```
Training observations: 793
Training groups: 750
Missing group identifiers: 0
```

In [77]:
```python
# cross-validation
# Five-fold group cross-validation
group_cv = GroupKFold(n_splits=5)

cv_results = cross_validate(
    linear_pipeline,
    X_train_group,
    y_train_group,
    groups=groups_train_lr,
    cv=group_cv,
    scoring={
        "MAE": "neg_mean_absolute_error",
        "RMSE": "neg_root_mean_squared_error",
        "R2": "r2"
    },
    n_jobs=-1
)

# Convert negative error scores into positive values
cv_mae = -cv_results["test_MAE"]
cv_rmse = -cv_results["test_RMSE"]
cv_r2 = cv_results["test_R2"]

print("LINEAR REGRESSION - 5-FOLD GROUP CV")
print("-" * 45)
```

```
print("Fold MAE:", np.round(cv_mae, 4))
print("Mean MAE:", round(cv_mae.mean(), 4))
print("Mean RMSE:", round(cv_rmse.mean(), 4))
print("Mean R²:", round(cv_r2.mean(), 4))
print("MAE standard deviation:", round(cv_mae.std(), 4))
```

```
LINEAR REGRESSION - 5-FOLD GROUP CV
--------------------------------------------
Fold MAE: [1.0647 1.2663 1.206  1.1794 1.2357]
Mean MAE: 1.1904
Mean RMSE: 2.2808
Mean R²: 0.4291
MAE standard deviation: 0.0692
```

In [78]:
```
# Build and tune Ridge
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

ridge_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", Ridge())
])

ridge_param_grid = {
    "regressor__alpha": [0.01, 0.1, 1, 10, 100, 500, 1000]
}

ridge_search = GridSearchCV(
    estimator=ridge_pipeline,
    param_grid=ridge_param_grid,
    scoring="neg_mean_absolute_error",
    cv=GroupKFold(n_splits=5),
    n_jobs=-1,
    verbose=1
)

ridge_search.fit(
    X_train_group,
    y_train_group,
    groups=groups_train_lr
)

print("Best Ridge parameters:")
print(ridge_search.best_params_)

print("\nBest Ridge CV MAE:")
print(round(-ridge_search.best_score_, 4))
```

```
Fitting 5 folds for each of 7 candidates, totalling 35 fits
Best Ridge parameters:
{'regressor__alpha': 0.1}

Best Ridge CV MAE:
1.1817
```

In [79]:
```
# Evaluate the best Ridge model
best_ridge = ridge_search.best_estimator_

y_pred_ridge = best_ridge.predict(X_test_group)

ridge_mae = mean_absolute_error(y_test_group, y_pred_ridge)
```

```
ridge_rmse = np.sqrt(mean_squared_error(y_test_group, y_pred_ridge))
ridge_r2 = r2_score(y_test_group, y_pred_ridge)

print("RIDGE REGRESSION - GROUPED TEST RESULTS")
print("-" * 45)
print(f"MAE  : {ridge_mae:.4f}")
print(f"RMSE : {ridge_rmse:.4f}")
print(f"R²   : {ridge_r2:.4f}")
```

```
RIDGE REGRESSION - GROUPED TEST RESULTS
---------------------------------------------
MAE  : 0.9826
RMSE : 1.4547
R²   : 0.4132
```

In [80]:
```
# step: 9 - Record Ridge's cross-validation result
print("Best Ridge parameters:", ridge_search.best_params_)
print("Best Ridge cv MAE:", round(-ridge_search.best_score_, 4))
```

```
Best Ridge parameters: {'regressor__alpha': 0.1}
Best Ridge cv MAE: 1.1817
```

In [81]:
```
# step: 10 - Build and tune Lasso Regression
from sklearn.linear_model import Lasso

lasso_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", Lasso(max_iter=20000, tol=0.001))
])

lasso_param_grid = {
    "regressor__alpha": [0.0001, 0.001, 0.01, 0.1, 1]
}

lasso_search = GridSearchCV(
    estimator=lasso_pipeline,
    param_grid=lasso_param_grid,
    scoring="neg_mean_absolute_error",
    cv=GroupKFold(n_splits=5),
    n_jobs=-1,
    verbose=1
)

lasso_search.fit(
    X_train_group,
    y_train_group,
    groups=groups_train_lr
)

print("Best Lasso parameters:")
print(lasso_search.best_params_)

print("\nBest Lasso CV MAE:")
print(round(-lasso_search.best_score_, 4))
```

```
Fitting 5 folds for each of 5 candidates, totalling 25 fits
Best Lasso parameters:
{'regressor__alpha': 0.001}

Best Lasso CV MAE:
1.2171
```

In [82]:
```
# Then evaluate
best_lasso = lasso_search.best_estimator_

y_pred_lasso = best_lasso.predict(X_test_group)

lasso_mae = mean_absolute_error(y_test_group, y_pred_lasso)
lasso_rmse = np.sqrt(mean_squared_error(y_test_group, y_pred_lasso))
lasso_r2 = r2_score(y_test_group, y_pred_lasso)

print("LASSO REGRESSION - GROUPED TEST RESULTS")
print("-" * 45)
print(f"MAE  : {lasso_mae:.4f}")
print(f"RMSE : {lasso_rmse:.4f}")
print(f"R²   : {lasso_r2:.4f}")
```

```
LASSO REGRESSION - GROUPED TEST RESULTS
---------------------------------------------
MAE  : 1.0932
RMSE : 1.8177
R²   : 0.0837
```

In [83]:
```
# Retrieve Ridge's cross-validation score
print("Best Ridge parameters:", ridge_search.best_params_)
print("Best Ridge CV MAE:", round(-ridge_search.best_score_, 4))
```

```
Best Ridge parameters: {'regressor__alpha': 0.1}
Best Ridge CV MAE: 1.1817
```

In [ ]: